

Project ATHENA — Day 8

Theme: Back testing — Can Your Strategy Survive Reality?

"The market doesn't care how elegant your strategy is. It only cares how it would have performed."

Objective: Build the foundation of ATHENA's back testing engine and learn the principles that separate credible research from misleading results.

Time Required: ~40 minutes

Today's Big Idea

Yesterday, your strategy generated signals:

BUY

HOLD

SELL

That's useful.

But here's the problem:

A signal is not evidence.

Until you've tested it against historical data, it's just an idea.

Backtesting is the process of asking:

"If I had followed these rules in the past, what would have happened?"

What Is a Backtest?

Imagine it's January 2022.

You only know what was available **on that day**.

Your strategy decides:

BUY

The next day arrives.

You update your data.

The strategy decides again.

You repeat this process through history.

Notice what you're **not** doing:

Looking ahead to future prices.

That's the first principle of a valid backtest.

The Four Rules of Honest Backtesting

Rule 1 — Never Peek Into the Future

This mistake is called **look-ahead bias**.

Incorrect:

Today's strategy



Uses tomorrow's closing price

Correct:

Today's strategy



Uses only today's information

A strategy that cheats—even accidentally—can appear spectacular while being impossible to implement.

Rule 2 — Include Trading Costs

Suppose your strategy earns:

12%

Annual return.

After commissions, taxes, spreads, and slippage:

8%

That difference matters.

A strategy making hundreds of trades per year is much more sensitive to costs than one making a handful.

From the beginning, ATHENA should assume that trading isn't free.

Rule 3 — Separate Research From Evaluation

Professional quants typically split data into:

Historical Data

↓

Research (In-sample)

↓

Validation (Out-of-sample)

If you repeatedly tweak a strategy until it fits historical data perfectly, you've likely built something that memorizes the past rather than generalizes to the future.

Rule 4 — Judge the Strategy, Not the Best Trade

Imagine two strategies.

Strategy A

- Makes ₹100,000 on one exceptional trade.
- Loses money on most others.

Strategy B

- Makes steady gains.
- Experiences controlled drawdowns.
- Produces consistent returns.

Which would you rather trust?

Almost always Strategy B.

Consistency is far more valuable than occasional brilliance.

Building ATHENA's Backtesting Engine

Our first version doesn't need to be complex.

Think in terms of responsibilities.

Price Data



Indicators



Strategy



Signal Generator



Trade Simulator



Performance Metrics

Each component should do one job well.

Designing the Engine

Create:

src/backtesting/

backtester.py

Define a class like:

```
class Backtester:

    def run(
        self,
        prices,
        strategy,
        initial_capital=100000,
        commission=0.001,
    ):
        """
        Simulate trades and return results.
        """
```

Don't implement every detail today.

Focus on a clean interface.

What Should the Backtester Return?

Instead of a single number, return structured results.

For example:

BacktestResult

- *Total Return*
- *Annual Return*
- *Number of Trades*
- *Win Rate*
- *Max Drawdown*

- *Final Portfolio Value*

- *Trade History*

Designing this well now will save you refactoring later.

Practical Exercise (≈40 Minutes)

Part 1 — Build the Framework (20 min)

Create:

src/backtesting/

backtester.py

results.py

Define:

class BacktestResult:

...

and

class Backtester:

...

Focus on method signatures, documentation, and clear responsibilities.

Part 2 — Simulate One Trade (15 min)

Without worrying about moving averages yet:

Create a tiny synthetic dataset.

Example:

Day 1

BUY

₹100

↓

Day 5

SELL

₹110

Your engine should calculate:

- Profit
- Return
- Updated portfolio value

One correctly simulated trade is better than a hundred lines of unfinished logic.

Part 3 — Journal (5 min)

Write down:

What assumptions is my backtester making that might not hold in real markets?

Examples might include:

- Instant execution.
- Unlimited liquidity.
- Fixed commissions.
- No market impact.
- Ability to buy fractional shares (if applicable).

Identifying assumptions early makes future improvements much easier.

Engineering Principle of the Day

Separate **strategy logic** from **backtesting logic**.

A strategy should answer only:

Should I Buy?

Should I Sell?

Should I Hold?

The backtester answers:

What happened because of those decisions?

Keeping those responsibilities separate allows you to test many strategies with the same engine.

Review Question

Suppose your strategy buys a stock whenever tomorrow's closing price is higher than today's.

It performs extraordinarily well in a backtest.

What's the most likely explanation?

- A. You've discovered an exceptional trading strategy.
- B. The strategy suffers from look-ahead bias because it uses future information.
- C. Moving averages are unnecessary.

Answer: B.

Any strategy using information that wasn't available at the time of the decision is invalid.

ATHENA Progress So Far

You've now designed the foundations of:

- ☒ Project architecture
- ☒ Risk management
- ☒ Return calculations
- ☒ Moving averages
- ☒ Strategy interface
- ☒ Backtesting framework

Notice a pattern: every component is modular. That's intentional. By the time we connect live market data, you'll already have a platform that resembles a professional research environment rather than a collection of notebooks.